

# Unit 4

# Memory and Basic Processor Interfaces

---

ECE-GY 6463: ADVANCED HARDWARE DESIGN

PROFS. SUNDEEP RANGAN, SIDDHARTH GARG

# Learning Objectives

| Skill                                | You will be able to                                                                                           |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Size memory for a data stream        | Compute data rate, capacity and the number of memories, and choose registers, block memory or external memory |
| Compute memory addresses             | Convert between word and byte addresses, split a global address into a local one, and lay out a register map  |
| Identify bus manager and subordinate | Say which party initiates a transaction, whichever way the data flows                                         |
| Time a valid/ready handshake         | Work out the cycle a transfer happens on, including when the receiver is busy                                 |
| Analyze AXI4-Lite transactions       | Time reads and writes across the AW, W, B, AR and R channels                                                  |
| Build an IP with AXI4-Lite registers | Write the Vitis HLS signature and s_axilite pragmas, and state the registers they produce                     |
| Determine the limiting resource      | Compare the time spent moving data against the time spent computing                                           |

Each skill is assessed by problems in the LLM grader.

# Outline

---

- ➡ Random Access Memory and Memory-Mapped Registers
  - ❑ Processor Interfaces with AXI4-Lite
  - ❑ Implementing AXI4-Lite Interface in Vitis HLS

# Addressable Memory

- An array of **elements** in storage
  - Use for vectors, matrices, packets, data, ...
- **Word**: Each data element
  - Typically, has **word / data width** = 32 or 64 bits
- **Memory depth** = number of words
- **Memory size**=depth  $\times$  word width
- **Address**: A location of a word
  - Often byte-addressable
  - Word  $k$  is at byte address  $4k$  when word width=32

| Byte address | Data       |
|--------------|------------|
| 0            | Word 0     |
| 4            | Word 1     |
| 8            | Word 2     |
| $\vdots$     | $\vdots$   |
| $4(N-1)$     | Word $N-1$ |

A 32-bit memory,  $N$  words deep

# Byte vs. Word Addressing

- **Word address:** counts words: 0, 1, 2, ...
- **Byte address:** counts bytes
  - A  $W$ -bit word spans  $W/8$  byte addresses
  - Word  $k$  starts at byte  $4k$  (32-bit) or  $8k$  (64-bit)
- A byte address splits into two fields
  - Low  $\log_2(W/8)$  bits: the byte within the word
  - Remaining bits: the word index
- Processor buses, including AXI, use **byte addresses**
  - even when every access is a whole word

32-bit memory, byte-addressable

| Word | Byte addresses |        |        |        |
|------|----------------|--------|--------|--------|
| 0    | 0              | 1      | 2      | 3      |
| 1    | 4              | 5      | 6      | 7      |
| 2    | 8              | 9      | 10     | 11     |
|      | ⋮              |        |        |        |
| $k$  | $4k$           | $4k+1$ | $4k+2$ | $4k+3$ |

A 32-bit byte address

| word index<br>bits 31–2 | byte<br>1–0 |
|-------------------------|-------------|
|-------------------------|-------------|

e.g.  $0x14 \rightarrow$  word 5, byte 0

# Byte and Word Addresses

PRACTICE

A memory stores 64-bit words and is byte-addressable: byte address 0 is the first byte of word 0, and each word occupies consecutive byte addresses.

- (a) What is the byte address of word 13? Give it in decimal and in hexadecimal.
- (b) Which word contains byte address `0x5C`, and at which byte offset within that word?
- (c) The memory has a depth of 4096 words. How many address bits are needed with byte addressing? How many with word addressing?

64-bit words: 8 bytes each

|   |   |   |    |    |    |    |    |    |
|---|---|---|----|----|----|----|----|----|
| 0 | 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  |
| 1 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
| ⋮ |   |   |    |    |    |    |    |    |

Word 13 starts at byte ?

Also in the LLM grader: Unit 4 › Byte and word addresses

# Byte and Word Addresses

SOLUTION

(a) 8 bytes per word, so word 13 starts at  $13 \times 8 = 104 = 0x68$

(b)  $0x5C = 92 = 8 \times 11 + 4 \rightarrow$  **word 11, byte 4**

word = `addr >> 3`    byte = `addr & 0x7`

(c) Word addressing:  $4096 = 2^{12}$  words  $\rightarrow$  **12 bits**

Byte addressing:  $4096 \times 8 = 2^{15}$  bytes  $\rightarrow$  **15 bits**

The extra 3 bits select the byte within the word

A byte address, 64-bit words

| word index<br>bits 14–3 | byte<br>2–0 |
|-------------------------|-------------|
|-------------------------|-------------|

## Common error

Using 4 bytes per word, the 32-bit habit, gives 52, word 23 and 14 bits.

Also in the LLM grader: Unit 4 › Byte and word addresses

# Sequential vs Random Access

---

## ■ Sequential Access

- Access time depends on *position*
- You must move through data in order
- Ex: Hard disks (seek + rotation)
- FIFO queues (logical sequential access)



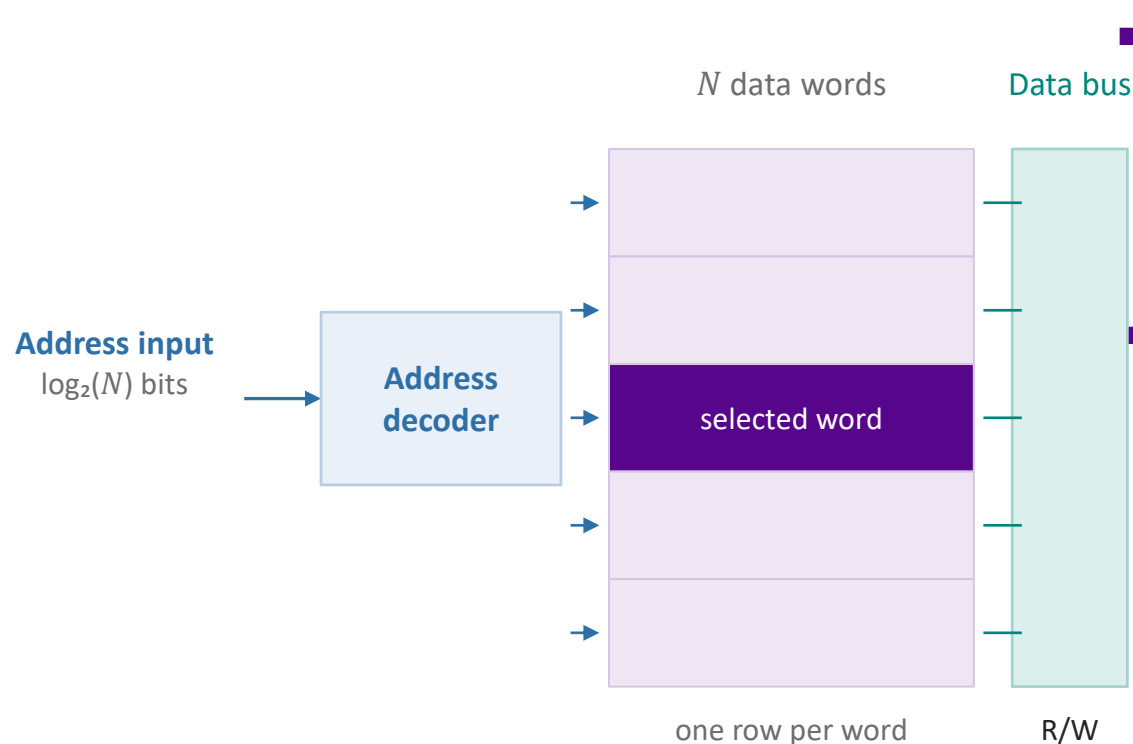
## ■ Random access

- Any element can be accessed in constant time
- SRAM (on chip block memory, FPGA BRAM)
- DRAM (external memory)
- Register files (architecturally random access)





# Typical RAM Architecture



## ■ Three interface signals:

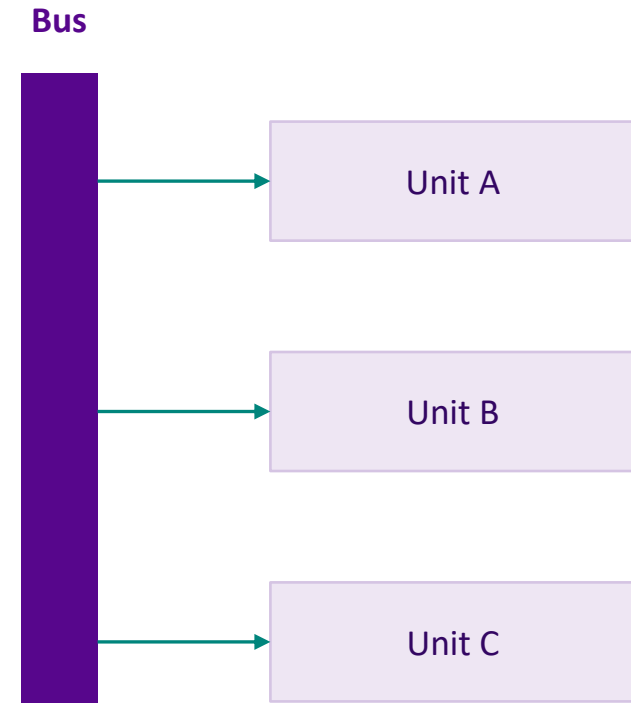
- **Address** input: Which word to select
- **Read/Write**: Which direction of transfer
- **Data**: Value to read or write

## ■ Address input:

- $\log_2(N)$  bits to select one of  $N$  words
- Each word has a wordline select
- Selection determined by **address decoder**

# Bus

- **Bus:** A **shared** communication pathway for multiple units
  - Physically a set of wires
  - For data, addresses, and control
  - A shared **protocol** so only one unit drives the bus at a time
- Many examples:
  - Multiple memory units connected to a common data bus
  - A processor and multiple memory units sharing address or data
  - Multiple peripherals on an SoC sharing an **AXI bus**
  - Multiple devices on an **I<sup>2</sup>C** or **SPI bus**
  - ...



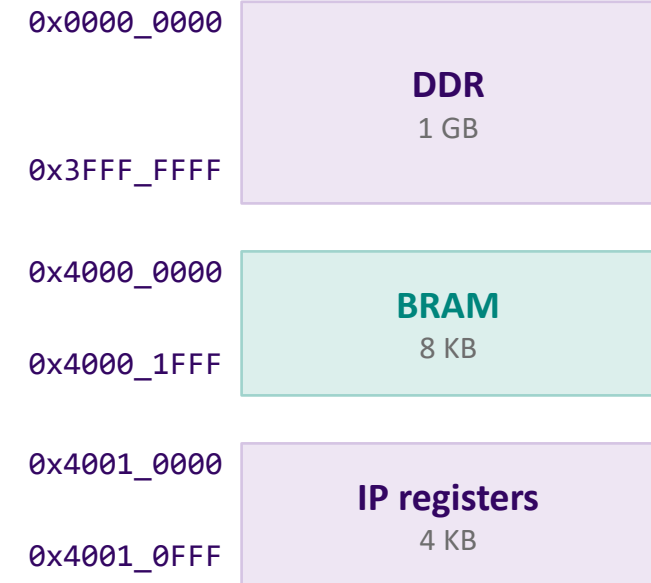
One unit drives the bus at a time

# Address Maps

- Modules on a shared bus share one **global address space**
- Each module gets a **base address** and a size
  - Size a power of two; base aligned to the size
- **Local address**: the offset within a module
  - Global address = base + local address
  - The module's decoder sees only the local address
- Decoding a global address
  - Upper bits select the module
  - Lower bits are the local address

Vitis also says “global memory” for off-chip DDR, a different meaning.

Example address map (not to scale)



Global 0x4000\_0104 → **BRAM**  
local = 0x4000\_0104 - 0x4000\_0000 = 0x104

# Global and Local Addresses

## PRACTICE

A processor bus uses 32-bit byte addresses. Three modules share it, each occupying a contiguous range of the global address space that starts at its base address.

- (a) What global address does the processor use to access word 100 of Memory B?
- (b) The processor reads global address `0x0000_2A0C`. Which module responds? What is the local byte address, and which local word is it?
- (c) What is the highest global address that belongs to Memory B? How many bits does Memory B need for its local byte address?

| Module       | Base address             | Size  |
|--------------|--------------------------|-------|
| Memory A     | <code>0x0000_0000</code> | 16 KB |
| Memory B     | <code>0x0001_0000</code> | 64 KB |
| IP registers | <code>0x0002_0000</code> | 256 B |

Both memories store 32-bit words.  
1 KB = 1024 bytes.

**Local address:** offset from the module's base address

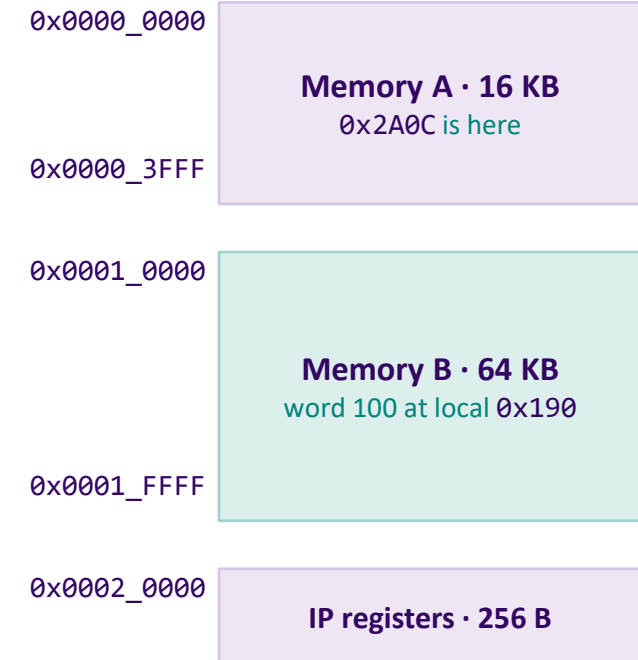
**Global address** = base + local address

Also in the LLM grader: Unit 4 › Global and local addresses

# Global and Local Addresses

SOLUTION

- (a) Word 100  $\rightarrow$  local byte  $100 \times 4 = 400 = 0x190$   
Global:  $0x0001\_0000 + 0x190 = 0x0001\_0190$
- (b) Memory A spans  $0x0000\_0000 - 0x0000\_3FFF$  (16 KB =  $0x4000$ )  
 $0x2A0C$  falls inside  $\rightarrow$  **Memory A**, local byte  $0x2A0C$  (10764)  
Local word =  $0x2A0C / 4 = 0xA83 = 2691$
- (c) 64 KB =  $0x1\_0000$  bytes  $\rightarrow$  B ends at  $0x0001\_FFFF$   
Local byte addresses  $0x0000 - 0xFFFF \rightarrow 16$  bits

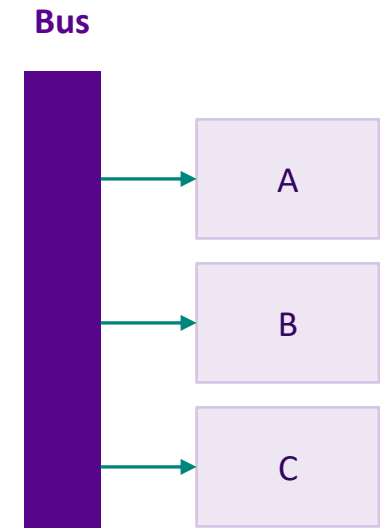


Also in the LLM grader: Unit 4 › Global and local addresses

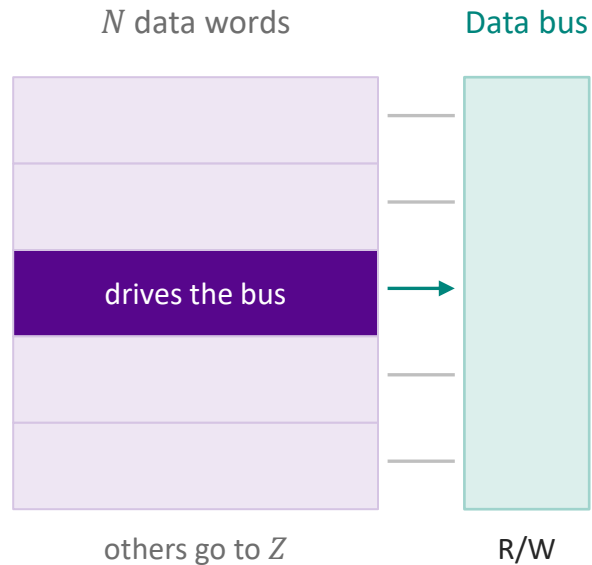
# High Impedance

- Method to share a common bus
  - One unit drives a value  $V$  on the bus
  - Other units go into a **high-impedance state**, denoted  $Z$
  - High impedance units are disconnected
  - Bus value =  $V$
- Sometimes called **Tri-State**
  - Each bit is 0, 1 or  $Z$

| A  | B  | C | Bus Value | Remarks                                                                      |
|----|----|---|-----------|------------------------------------------------------------------------------|
| Z  | 15 | Z | 15        | B drives bus                                                                 |
| 41 | Z  | Z | 41        | A drives bus                                                                 |
| Z  | Z  | Z | X         | No driver- no reliable value                                                 |
| 32 | 78 | Z | X         | Collision of A, B – no reliable value<br>Protocols will avoid this condition |

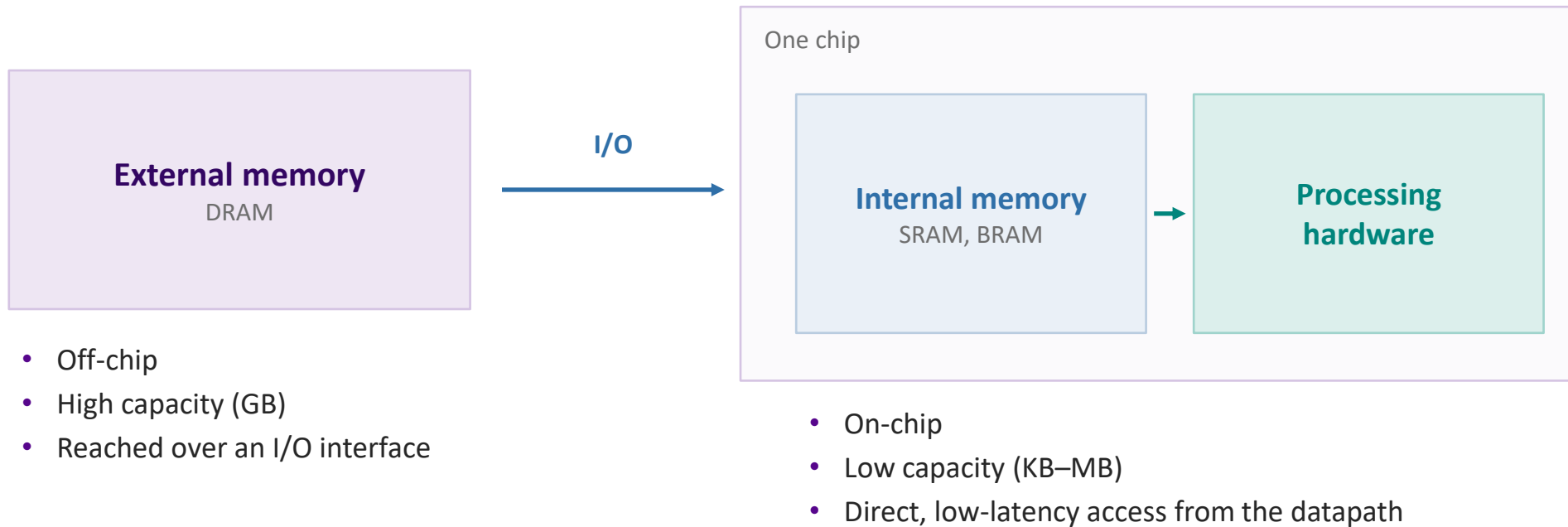


# High Impedance on a Data Bus



- **Memory read** (Tri-State Bus)
  - Selected memory element drives its value
  - Other units go to  $Z$
  - Data bus value = selected memory element's value
- **Memory write** (Broadcast + Selective Capture)
  - Data bus value appears at all units
  - Selected element loads value into its storage element
  - Other elements ignore value

# External vs. Internal Memory

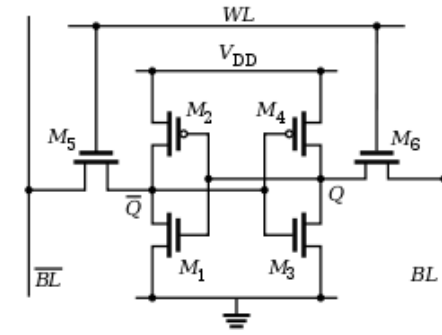




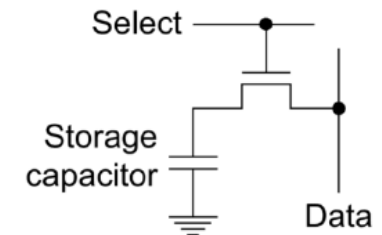
# External vs. Internal Compared

| Feature      | Internal memory (SRAM)             | External memory (DRAM)        |
|--------------|------------------------------------|-------------------------------|
| Location     | On-chip                            | Off-chip                      |
| Storage cell | 6-T cell (cross-coupled inverters) | Capacitor + access transistor |
| Density      | Low                                | High                          |
| Capacity     | Small (KB–MB)                      | Large (GB)                    |
| Speed        | Very fast                          | Slower                        |
| Power        | Higher static                      | Lower static, higher dynamic  |
| Access       | Direct by logic                    | Through a memory controller   |
| Refresh      | No                                 | Yes                           |

SRAM cell



DRAM cell



# Example On-Chip Memory Sizes

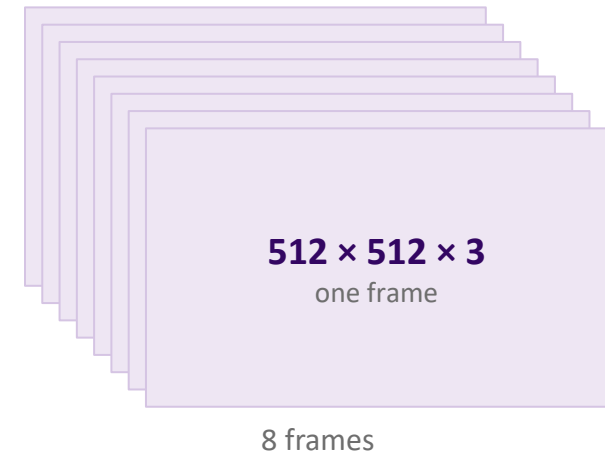
| Platform                                | On-chip memory      | Typical size | Notes                                    |
|-----------------------------------------|---------------------|--------------|------------------------------------------|
| Laptop CPU (Intel Core)                 | SRAM (L1/L2/L3)     | 20–40 MB     | L3 dominates; very fast                  |
| NVIDIA A100 GPU                         | SRAM (L2 cache)     | ~40 MB       | HBM2e (40–80 GB) is off-chip, on-package |
| Xilinx Virtex-7 FPGA                    | BRAM (36 Kb blocks) | 2–6 MB       | Distributed across the fabric            |
| Custom ASIC (5 mm <sup>2</sup> , 16 nm) | SRAM                | 5–10 MB      | Depends on layout and redundancy         |

**Lesson: internal memory is limited. Use it judiciously.**

A 16 nm 6-T high-density bitcell is about 0.07  $\mu\text{m}^2$  (TSMC 128 Mb SRAM, ISSCC 2014), so 5 mm<sup>2</sup> of bitcells holds  $\approx 70$  Mbit — roughly 5–7 MB once decoders and sense amplifiers are counted.

# Example: Video

- A video processor stores 8 frames of raw video
  - $512 \times 512$  pixels, 3 color channels, 8 bits per channel
- Memory capacity
  - $B = 512 \times 512 \times 3 \times 8$  bytes =  $3 \cdot 2^{21}$  bytes  $\approx$  **6.3 MB**
  - A sizeable portion of memory, even on a large chip
- Stored as 32-bit words
  - $N = B/4 = 3 \cdot 2^{19} \approx 1.57$  M words
- Address bits: the smallest  $K$  with  $2^K \geq$  the count
  - Byte addressing:  $2^K \geq 3 \cdot 2^{21} \rightarrow$   **$K = 23$**
  - Word addressing:  $2^K \geq 3 \cdot 2^{19} \rightarrow$   **$K = 21$**



**6.3 MB**

capacity

**23 bits**

byte address

# Registers vs. Bulk Storage

Every IP holds a mix. The choice is made per value, not per design.

## Registers

- Parallel access: many values in the same cycle
- Any register value is available to the datapath
- Higher area per bit
- A host reads and writes them one at a time

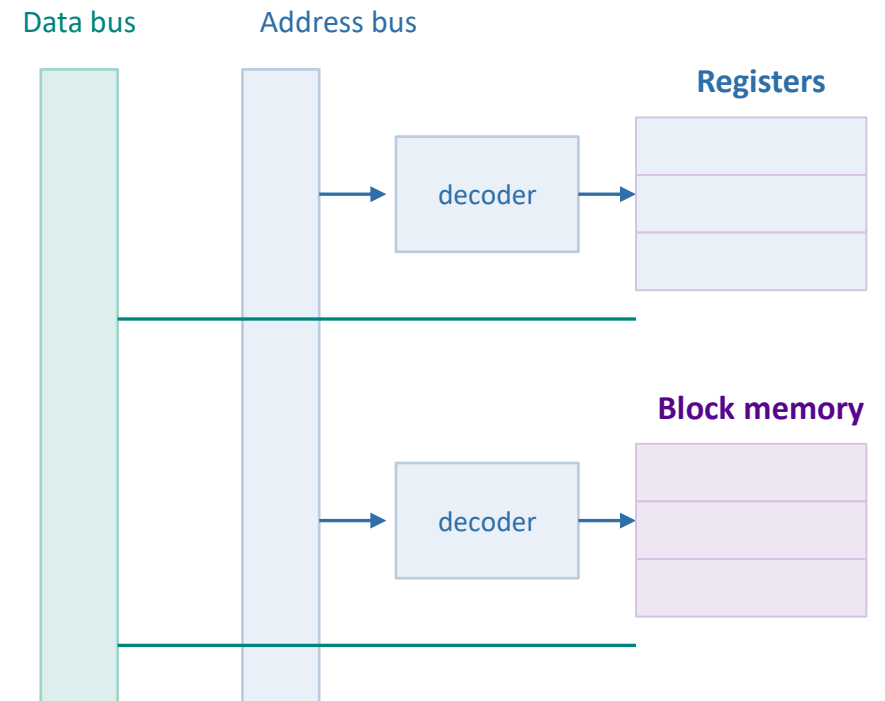
## Bulk storage

- Typically one word per access
- Much higher density
- A host moves data to and from it in bulk
- Block RAM on chip, DRAM off chip

A few values needed at once → **registers**. Thousands touched one at a time → **bulk storage**.

# Memory-Mapped Storage

- Registers can also be memory-mapped
  - Connected to their own address decoder
  - Determines which register to read/write
- Common interface to processor
  - Processor sees registers and block memory in one address space
  - Being addressable is independent of what the storage is made of



# Example: The Cubic IP

- A HW module computes a cubic function:

$$y_i = a_0 + a_1x_i + \dots + a_3x_i^3$$

- Registers:

- Store polynomial coefficients  $a[0], \dots, a[3]$
- Four values, all needed in the same cycle
- Want parallel access
- Could have control registers, e.g. count, data valid, ready, ...

- Block memory

- Store input and output data
- $x[0], \dots, x[n-1]$  and  $y[0], \dots, y[n-1]$
- n values, only one touched per cycle

Registers

|        |
|--------|
| $a[0]$ |
| $a[1]$ |
| $a[2]$ |
| $a[3]$ |

|          |          |
|----------|----------|
| $x[0]$   | $y[0]$   |
| $x[1]$   | $y[1]$   |
| $\vdots$ | $\vdots$ |
| $x[n-1]$ | $y[n-1]$ |

Block memory

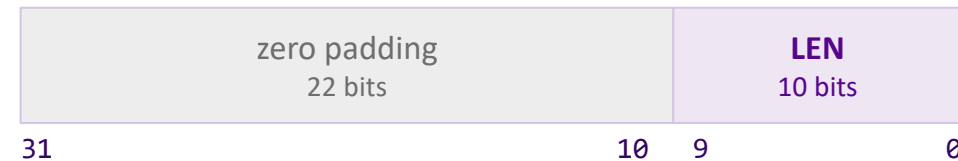
# Register Maps

- **Register map**: where each field lives
  - Offset from the IP's base, name, field bits, access
- For now: no field spans two registers
  - Each field fits inside one 32-bit register
  - Tight packing comes later, with serialization
- Unused upper bits are **zero padding**
  - Read back as 0; ignored by the hardware

Example: a vector-scaling IP

| Offset | Register | Field bits          | Access |
|--------|----------|---------------------|--------|
| 0x00   | CTRL     | start [0], done [1] | R/W    |
| 0x04   | GAIN     | 7:0                 | R/W    |
| 0x08   | LEN      | 9:0                 | R/W    |
| 0x0C   | SUM      | 23:0                | R      |

The **LEN** register: 32 bits



# Register Map of the Cubic IP

PRACTICE

The cubic IP computes  $y[i] = a_0 + a_1x[i] + a_2x[i]^2 + a_3x[i]^3$  over a vector of length  $n$ . Its registers hold:

- **start, done**: 1 bit each, sharing CTRL (start = bit 0, done = bit 1)
- **a0–a3**: 16-bit signed coefficients
- **n**: 12-bit unsigned vector length

- (a) Write the register map: offset, field(s) and zero-padding bits for each register.
- (b) The IP is mapped at base **0x4000\_0000**. What global address does the processor write to set **a2**?
- (c) The processor writes  $a_2 = -3$ . What 32-bit value reads back, in hex? What number does code get if it treats that as a signed 32-bit integer?

## Layout rules

32-bit registers, one field each (CTRL holds both bits)  
Fields in the low bits, zero padding above  
Order from 0x00: CTRL, a0, a1, a2, a3, n

| Offset | Register | Field bits | Padding |
|--------|----------|------------|---------|
|        | CTRL     |            |         |
|        | a0       |            |         |
|        | a1       |            |         |
|        | a2       |            |         |
|        | a3       |            |         |
|        | n        |            |         |

Also in the LLM grader: Unit 4 › Register map



# Register Map of the Cubic IP

SOLUTION

- (a) See the table: registers sit 4 bytes apart
- (b)  $0x4000\_0000 + 0x0C = 0x4000\_000C$
- (c)  $-3$  in 16-bit two's complement is  $0xFFFF$   
Zero padding  $\rightarrow$  reads back as  $0x0000\_FFFD$   
As a signed 32-bit integer: **65533**, not  $-3$

| Offset | Register | Field bits          | Padding |
|--------|----------|---------------------|---------|
| 0x00   | CTRL     | start [0], done [1] | 30      |
| 0x04   | a0       | 15:0                | 16      |
| 0x08   | a1       | 15:0                | 16      |
| 0x0C   | a2       | 15:0                | 16      |
| 0x10   | a3       | 15:0                | 16      |
| 0x14   | n        | 11:0                | 20      |

The **a2** register after writing  $-3$

0000 0000 0000 0000  
bits 31–16: padding

1111 1111 1111 1101  
bits 15–0:  $-3$

**Watch out:** zero padding does not sign-extend.  
Software must sign-extend a padded signed field itself.

Also in the LLM grader: Unit 4 › Register map

# Where Does Each Value Live?

IN CLASS

Pick a computation that interests you: filtering in video processing, encryption, a neural-network layer, a matrix product. It must take at least one array, image or stream.

- (a) List the inputs and outputs, with a size or a rate for each.
- (b) Which would you hold in **registers**? Why?
- (c) Which would you hold in **bulk storage**? Why?
- (d) Choose a problem size. Give the register map, and the size of the bulk storage in words.

## Decide per value

Registers: parallel access, available to the datapath, higher area per bit

Bulk storage: one word per access, much higher density

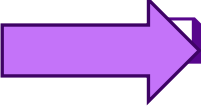
How the data crosses into the IP is the next section, and units 5 and 9.

Also in the LLM grader: Unit 4 › Storage partitioning

# Outline

---

- ❑ Random Access Memory and Memory-Mapped Registers

-  Processor Interfaces with AXI4-Lite

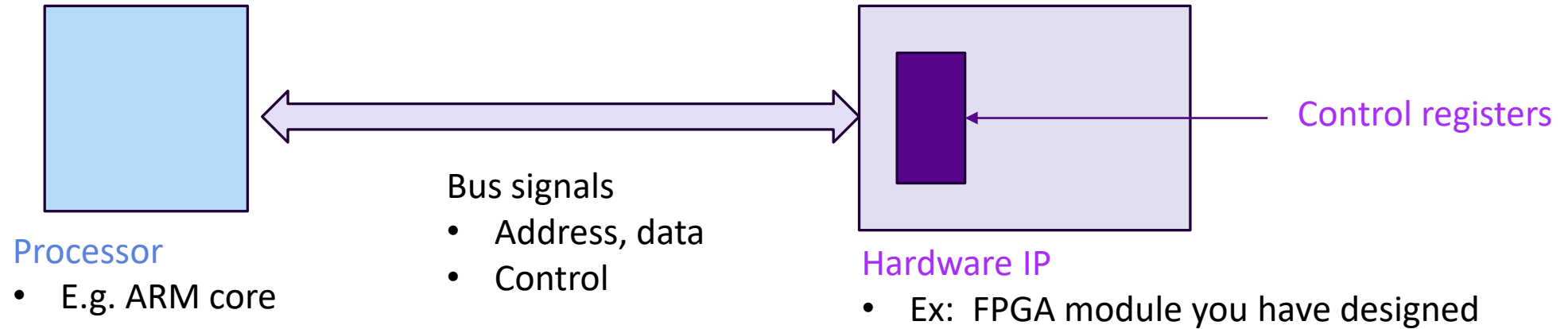
- ❑ Implementing AXI4-Lite Interface in Vitis HLS

# AXI4-Lite Protocol

---

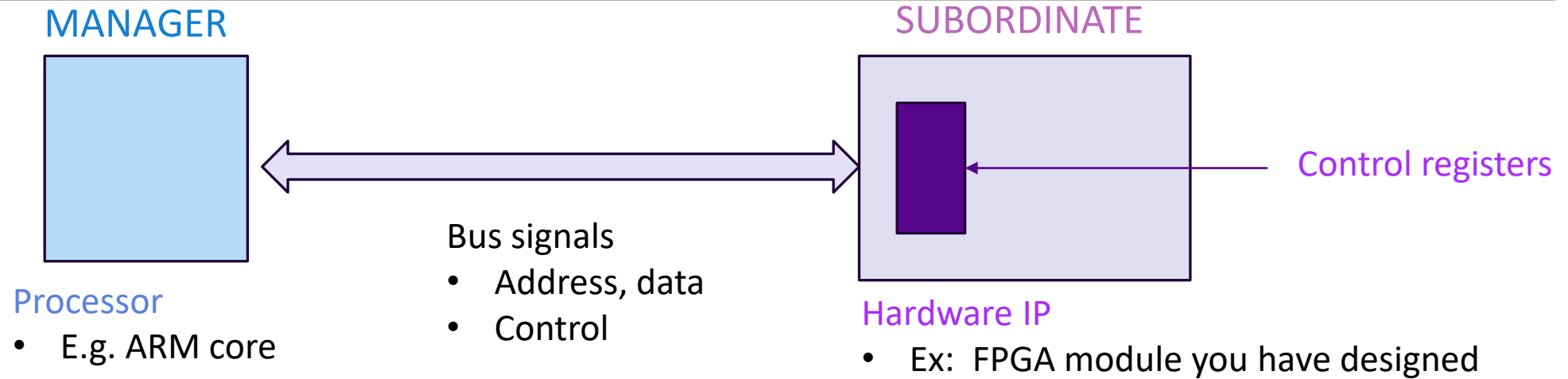
- ❑ ARM Extensible Interface family of protocols
  - Others we will use in class are AXI4-Full and AXI4-Stream
- ❑ Industry-standard, ubiquitously used
- ❑ AXI4-Lite:
  - Simple
  - Ideal for programming registers
  - Modular enables flexibility in delays and scheduling at both ends
  - Typical throughput: 1 read / 1 write every 3-5 clock cycles

# AXI4-Lite Use Case: Processor to Hardware IP



- ☐ Widely-used use case
- ☐ Hardware IP has control registers
- ☐ Need read / write access from an embedded processor

# Managers and Subordinates

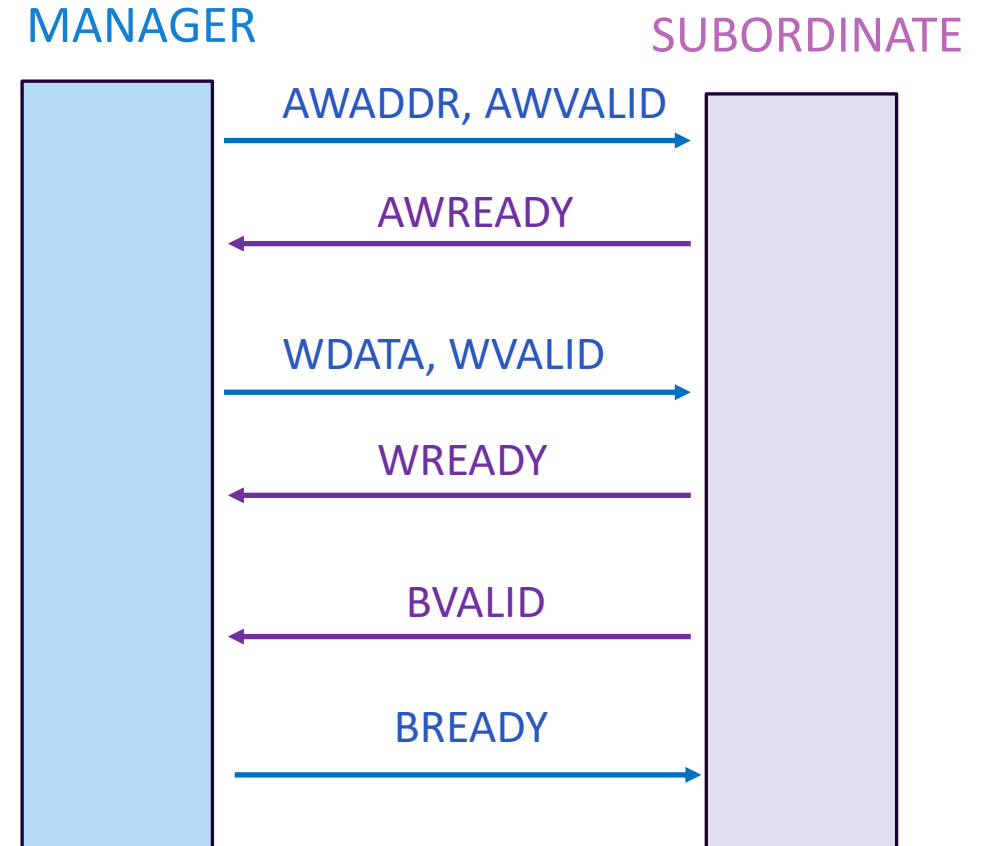


Called master and slave until AMBA renamed them in 2021, and the port names (S\_AXI, s\_axilite) still do. In AXI4-Lite:

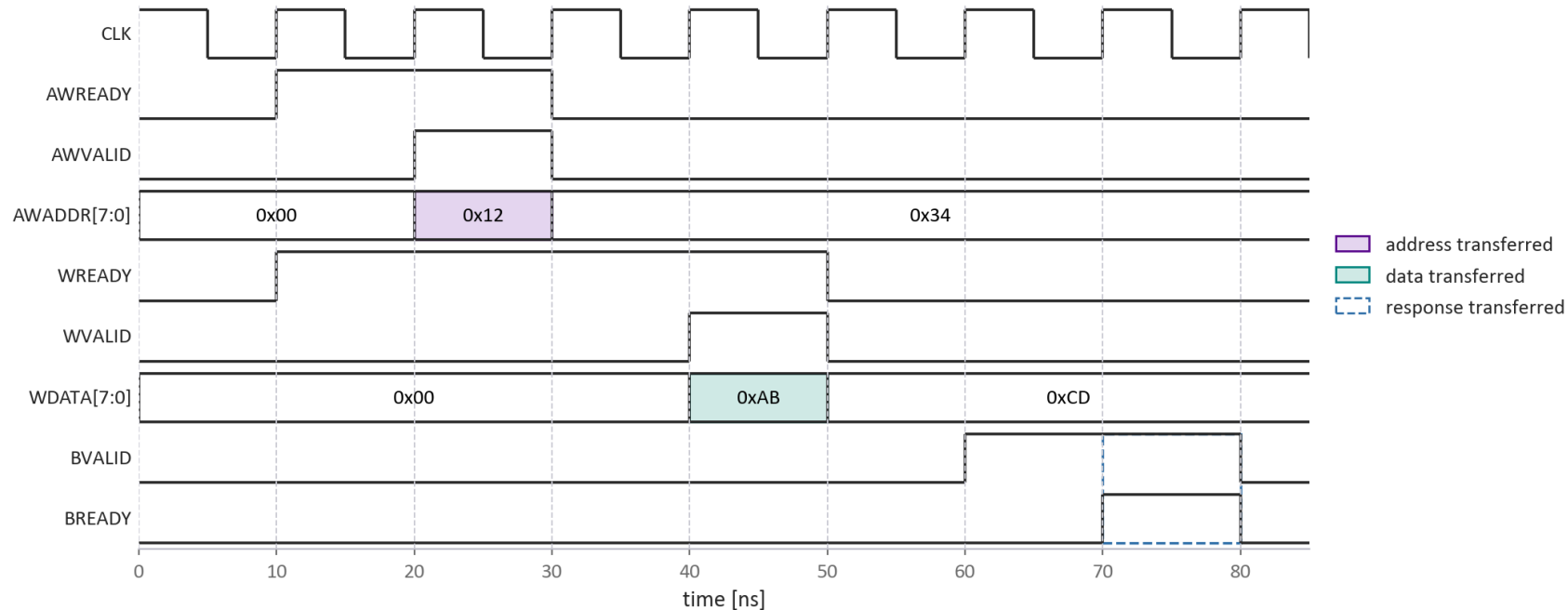
- ❑ Each read or write of a single word is a **transaction**
- ❑ **Manager**: The unit that initiates all transaction (typically the processor in this example)
- ❑ **Subordinate**: The unit that responds to the transaction (typically the hardware IP)

# Review: AXI4-Lite Write Protocol

- ❑ Suppose **manager** wants to write data  $D$  to address  $A$ 
  - $AWADDR=A$  and  $AWVALID=1$
  - $WDATA=D$  and  $WVALID=1$
  - Both can occur in independent clock cycles
- ❑ When **subordinate** is ready for address or data it sets:
  - $AWREADY=1$  and  $WREADY=1$
  - Subordinate can set these independently and any time it is ready
- ❑ Transfer:
  - When  $AWREADY=AWVALID=1$  together address  $A$  is transferred
  - When  $WREADY=WVALID=1$  together data  $D$  is transferred
- ❑ Write response (B channel):
  - After both transfers, the subordinate sets  $BVALID=1$  with the write response
  - The manager sets  $BREADY=1$  to accept it; the response transfers when  $BVALID=BREADY=1$
  - Each channel is free for the next transaction once its own handshake completes



# Example AXI4-Lite Write



Example with

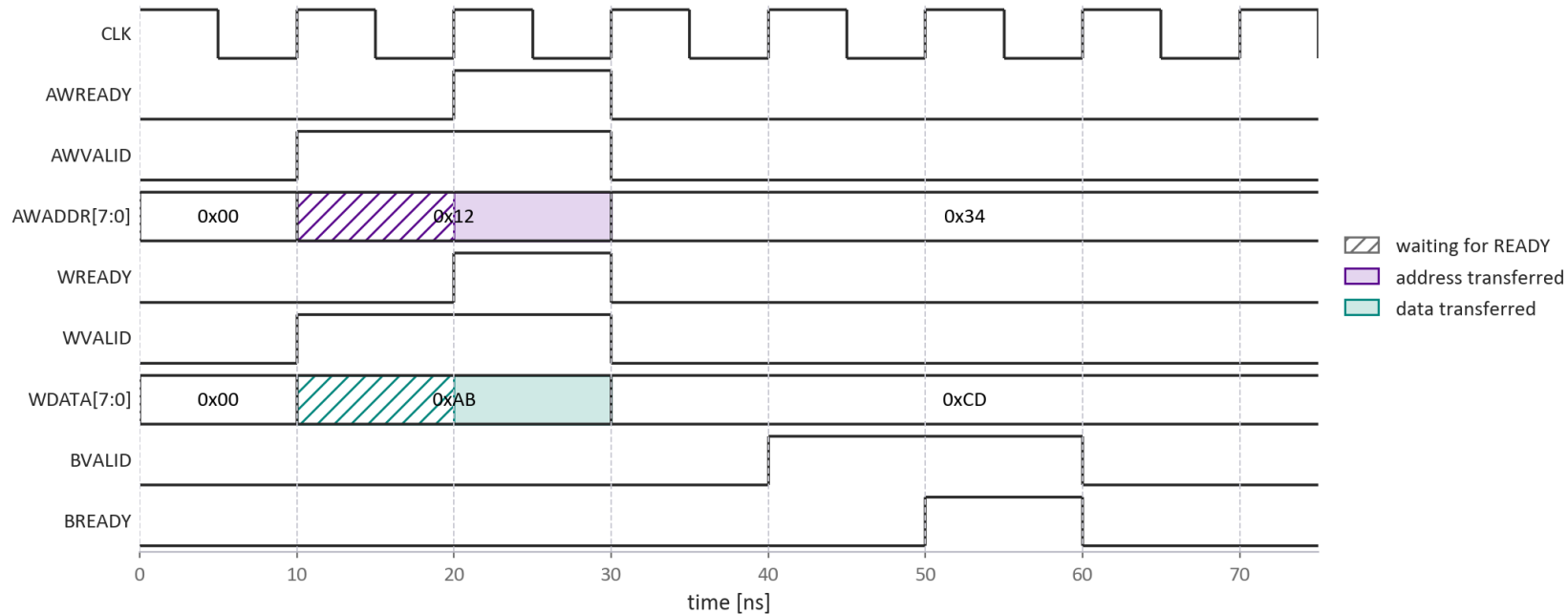
Subordinate is ready before manager

Address write before data write

But either order or together is fine



# Example AXI4-Lite Write With Stall



Example:

- Manager sets valid
- Subordinate is not ready
- Transfer stalls
- Provides flow control
  - Ensures data waits until RX is ready

# Read the Timing Diagram

IN CLASS

Using the write-with-a-stall diagram on the previous slide:

- (a) In which cycle does the address transfer on the AW channel? How do you know?
- (b) In which cycle does the write data transfer on the W channel?
- (c) In which cycle does the write response transfer on the B channel?
- (d) The hatched region marks a wait. Which side is waiting, what is it waiting for, and for how many cycles?

A transfer happens on the first cycle where **VALID** and **READY** are both high.

Also in the LLM grader: Unit 4 › Reading a write timing diagram

# Theoretical Optimal Write Throughput

---

## ❑ Cycle 0:

- Manager sets  $AWVALID=1$ ,  $AWADDR=A$  and  $WDATA=D$  and  $WVALID=1$
- Subordinate is ready:  $AWREADY=WREADY=1$
- Both address and data are transferred to subordinate

## ❑ Cycle 1:

- Subordinate set  $BVALID=1$  signaling transaction complete on subordinate side
- Subordinate registers

## ❑ Cycle 2:

- Manager acknowledges with  $BREADY=1$
- Manager starts next transaction in same clock cycle

❑ Theoretical optimal throughput = 1 word write / 2 clock cycles

❑ Theoretical minimum latency = 2 clock cycles (manager write + subordinate register)

# Practical AXI4-Lite Latency

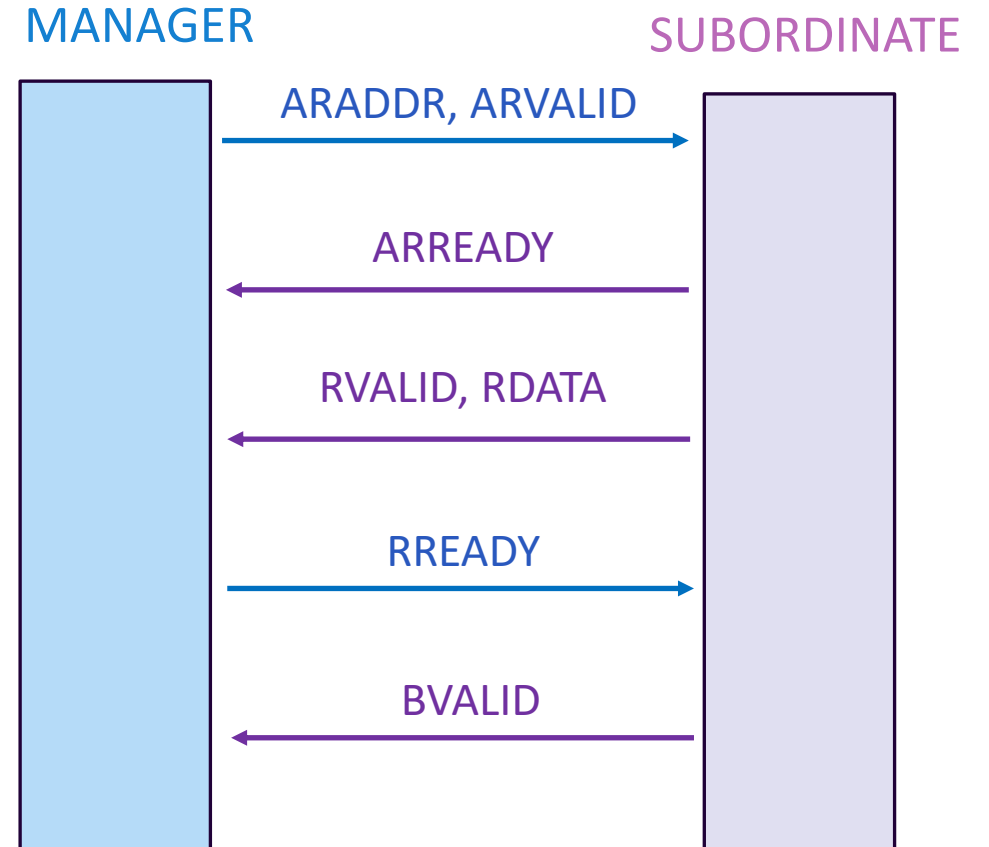
---

- ❑ Suppose:
  - Manager sets data and address on cycle 0
  - Subordinate is ready
- ❑ In most practical register systems:
  - Subordinate takes 1-2 additional clock cycles for BREADY=1
- ❑ Manager takes at 1-2 additional clock cycles for BVALID=1 and next input

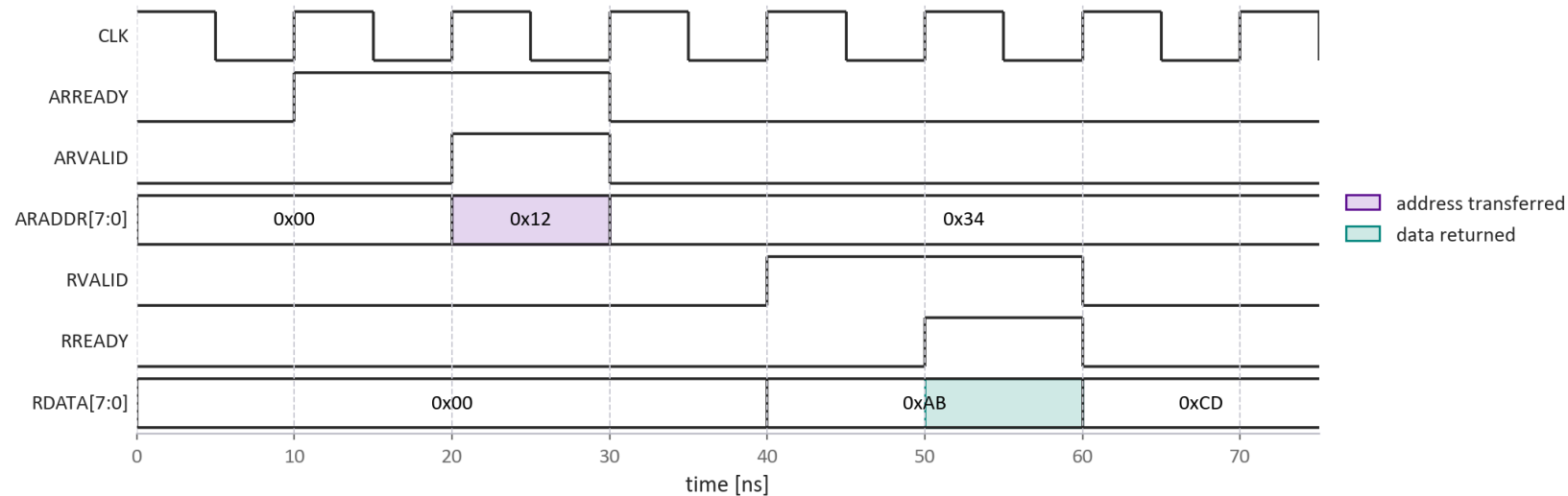
Practical throughput with AXI4-Lite: 1 word write per 3-5 clock cycles

# AXI4-Lite Read Protocol

- ❑ Suppose **manager** wants to read data from address  $A$ 
  - $ARADDR=A$  and  $ARVALID=1$
  - Indicates that it wants to read data
- ❑ When **subordinate** accepts the request:
  - Sets  $ARREADY=1$
- ❑ When subordinate has fetched data:
  - Sets  $RVALID=1$ ,  $RDATA=D$
  - Can be 0 to many cycles after  $ARREADY=1$
- ❑ Handshake:
  - Manager asserts  $RREADY=1$  when ready to accept data
  - $RREADY$  could be asserted before  $RVALID=1$
  - Transfer completes when  $RVALID=RREADY=1$



# Example Read Timing



In this example:

- ❑ Subordinate is initially ready (ARREADY=1)
- ❑ Subordinate immediately accepts request
- ❑ Manager is immediately available for
- ❑ Data valid with two cycle delay

# Theoretical Optimal Read Throughput

---

## ❑ Cycle 0:

- Manager sets  $ARADDR=A$  and  $ARVALID=1$
- Subordinate is ready:  $ARREADY=1$
- Address is transferred to subordinate

## ❑ Cycle 1:

- Subordinate set  $RVALID=1$  with data  $RDATA=D$
- Manager is immediately ready,  $RREADY=1$

## ❑ Cycle 2:

- Manager moves to next read address

❑ Theoretical optimal throughput = 1 word read / 2 clock cycles

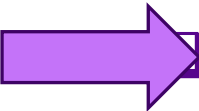
❑ Practical throughput = 1 word read per 3-5 clock cycles

# Outline

---

❑ Random Access Memory and Memory-Mapped Registers

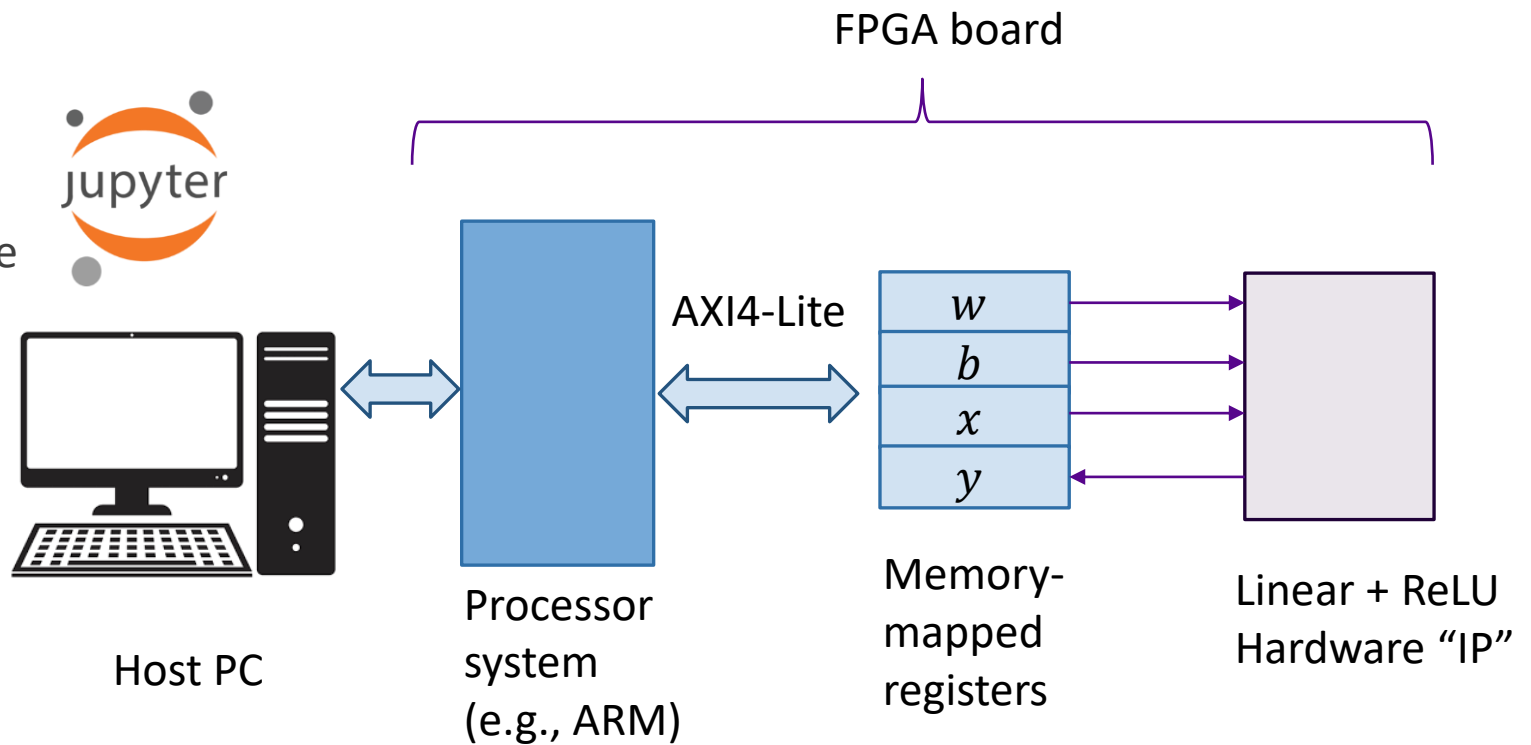
❑ Processor Interfaces with AXI-Lite

 Implementing AXI-Lite Interface in Vitis HLS



# Example: Connecting a Simple IP

- ReLU + Linear  $y = \max\{wx + b, 0\}$ 
  - Identical to Unit 1
- Add an interface to a processor
  - Processor can be an embedded
  - Ex: ARM Core on Pynq board
  - Interfaces to the IP over AXI4-Lite
- Add control from host PC
  - Ideally jupyter notebook



# HLS vs. RTL

- RTL: Register Transfer Language

- Use specialized languages (e.g., SystemVerilog)
- Manually schedule action in each clock cycle
- Interfaces designed from scratch
- ✓ – High control, supports custom interfaces
- ✗ – But time-consuming, difficult to maintain

- HLS: High-Level Synthesis

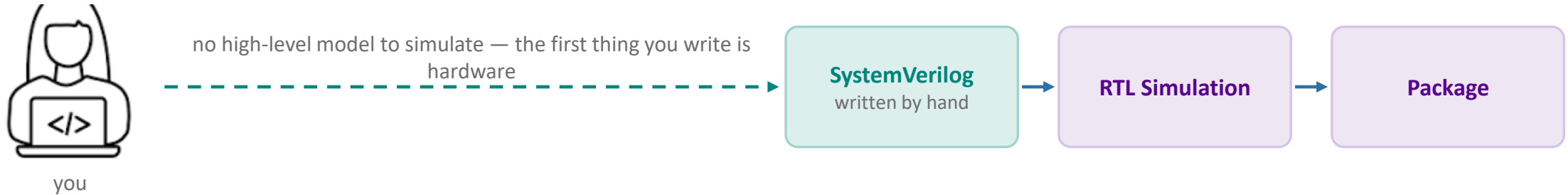
- Write in general purpose languages (e.g., C++)
- Clock-level schedule determined automatically by compiler
- API support for common interfaces (e.g., AXI)
- ✗ – Sacrifice some performance and flexibility
- ✓ – But enables rapid development

```
module relu_lin (  
    input logic int x,  
    input logic int a,  
    input logic int b,  
    output logic int y  
);  
always_comb begin  
    logic int mult_out, add_out;  
    mult_out = x * a;  
    add_out = mult_out + b;  
    y = (add_out > 0) ? add_out : 0;  
end  
endmodule
```

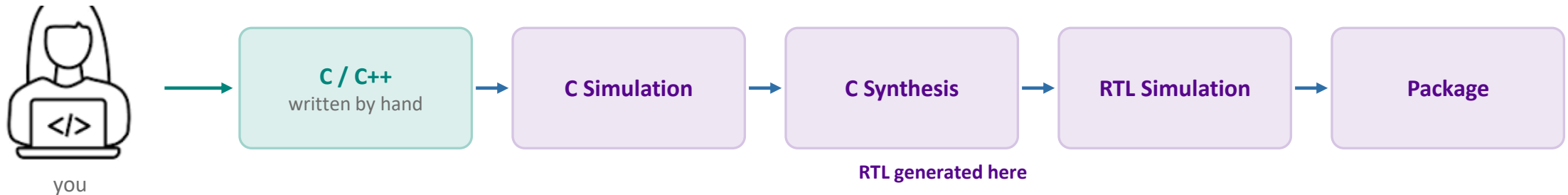
```
void relu_lin(int x, int a, int b, int &y) {  
    int mult_out = x * a;  
    int add_out = mult_out + b;  
    if (add_out > 0) {  
        y = add_out;  
    } else {  
        y = 0;  
    }  
}
```

# HLS vs. RTL Design Flow

## Traditional RTL — you write the hardware description



## HLS — you write behaviour, the tool writes the hardware



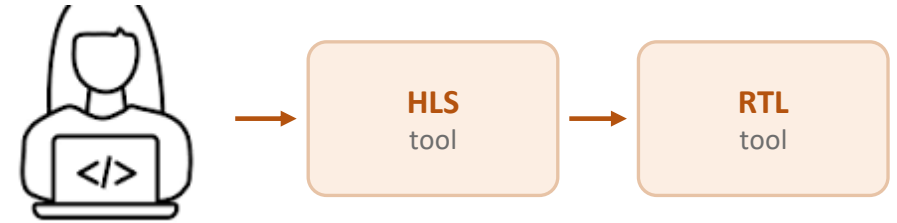
**One flow, two front ends** — the last stages are identical. HLS changes what you write, not what the FPGA gets.

# Bad vs. Good HLS Code

## AVOID

### Treat the tool as a black box

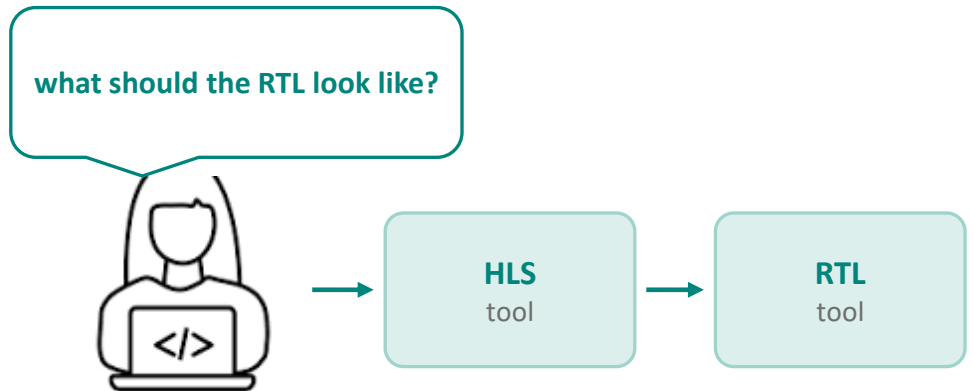
- You get RTL you cannot judge
- Poor performance, and no idea why



## AIM FOR

### Keep a mental model of the RTL

- What should happen on each clock cycle?
- What resources should it use?

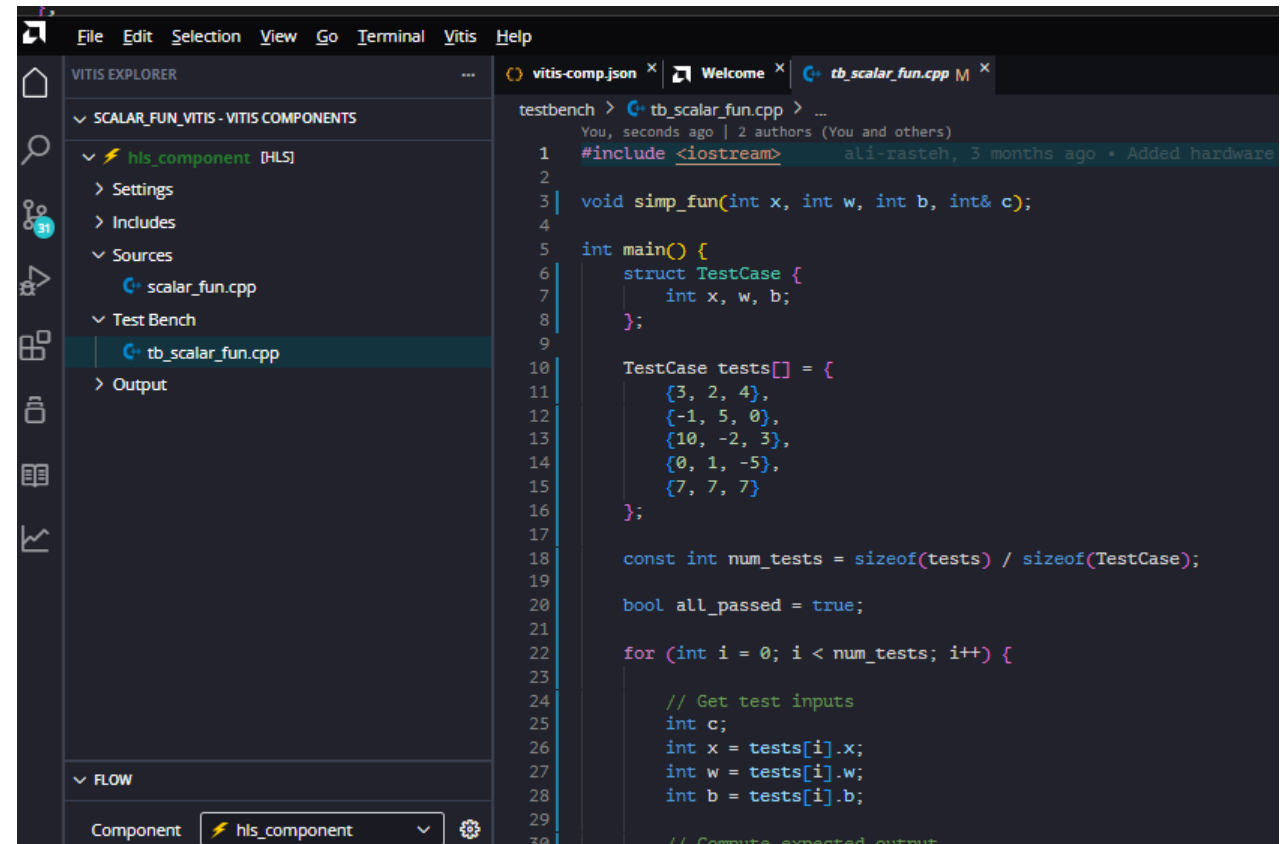


The tool schedules the cycles — but only you can tell whether what it produced is any good.

# Vitis HLS



- Developed by Xilinx (now AMD)
  - Evolved from Vivado HLS
- Supports C / C++
  - Simple, common development
- Some python support for testbenches
- Ships with Vitis IDE
- Native support for AXI protocols
  - No need to build handshake from scratch



```
testbench > tb_scalar_fun.cpp > ...
You, seconds ago | 2 authors (You and others)
ali-rasteh, 3 months ago • Added hardware

1  #include <iostream>
2
3  void simp_fun(int x, int w, int b, int& c);
4
5  int main() {
6      struct TestCase {
7          int x, w, b;
8      };
9
10     TestCase tests[] = {
11         {3, 2, 4},
12         {-1, 5, 0},
13         {10, -2, 3},
14         {0, 1, -5},
15         {7, 7, 7}
16     };
17
18     const int num_tests = sizeof(tests) / sizeof(TestCase);
19
20     bool all_passed = true;
21
22     for (int i = 0; i < num_tests; i++) {
23
24         // Get test inputs
25         int c;
26         int x = tests[i].x;
27         int w = tests[i].w;
28         int b = tests[i].b;
29
30         // Compute expected output
```

# FPGA Board Components

- We target a design on a real FPGA board
- Boards have three components
- **Processing system (PS)**
  - A lightweight processor
  - Often an ARM core
  - E.g., ARM A9
- **Programmable logic (PL)**
  - The programmable fabric
  - This is where your IP will reside
  - Also contains the registers and internal memory
- An **AXI4 interface**

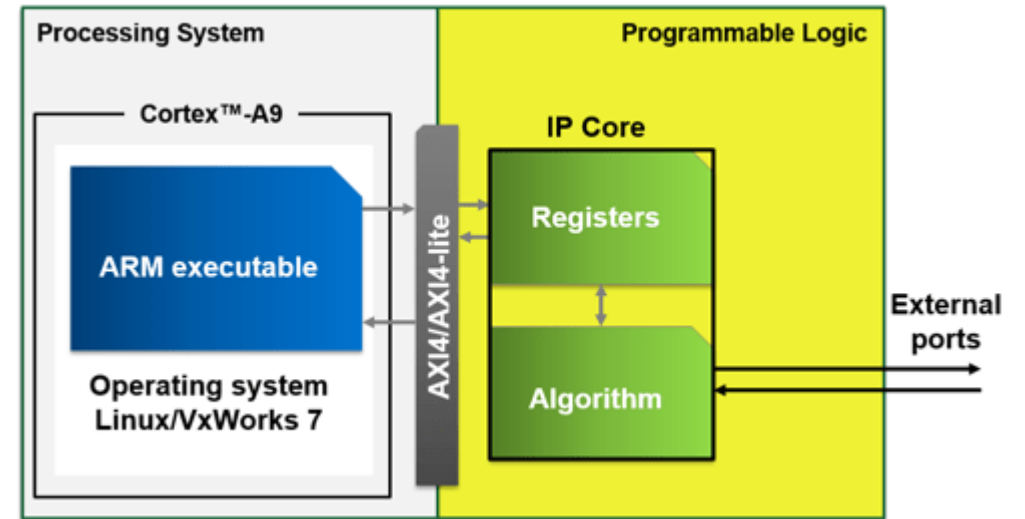
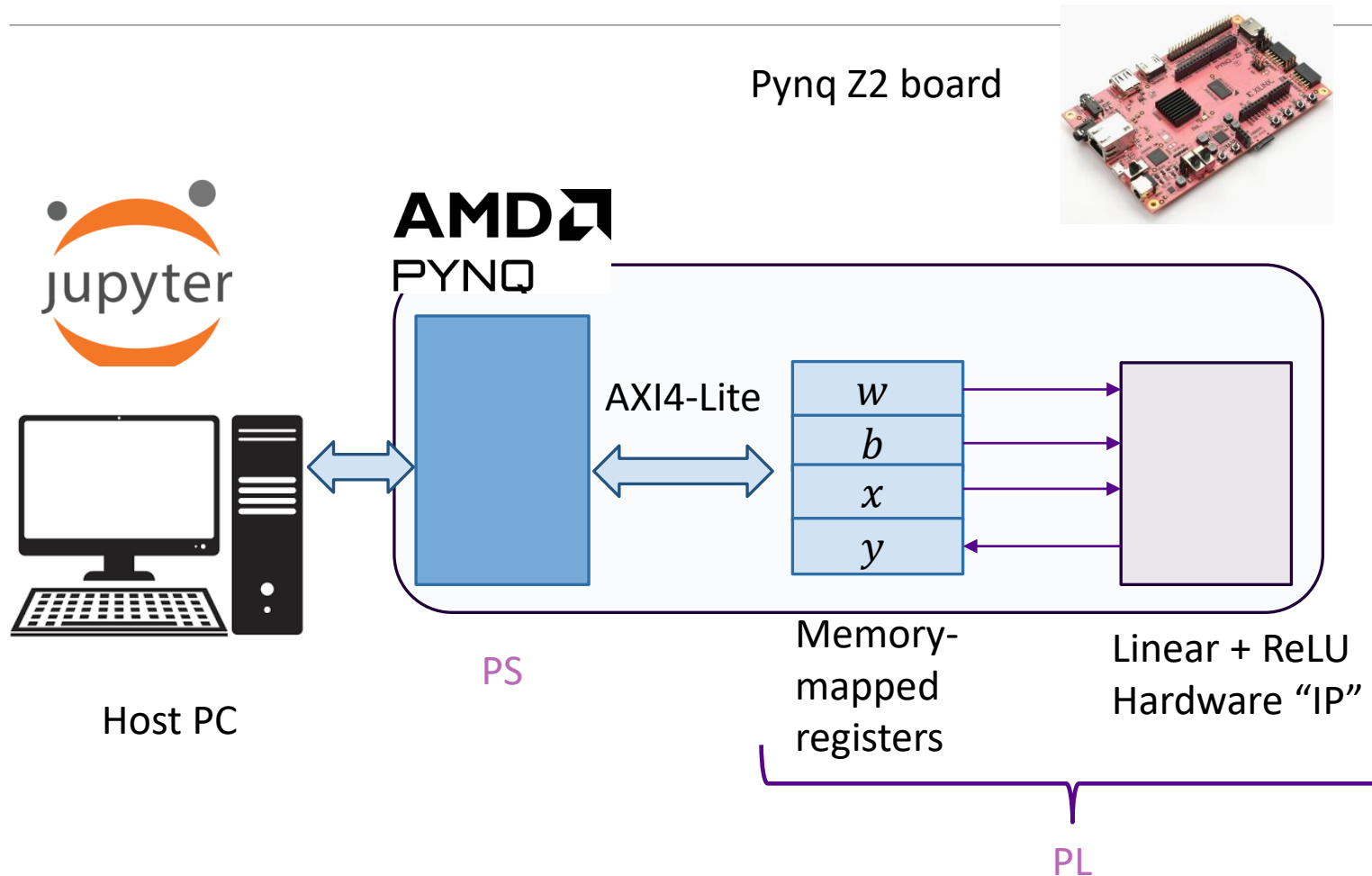


Image from MathWorks [page](#) on embedded design



# Target Architecture



We will rapidly build

- Complete AXI4-Lite I/F
  - Interface done automatically
  - No need to code any handshaking
- Control through host
  - Simple python jupyter notebook
- Deploy to a real FPGA!

# Vitis Kernel Types

Vitis kernels come in two shapes. The difference is who decides when the kernel runs.

## Host-Activated

```
#pragma HLS INTERFACE s_axilite port=return  
ap_ctrl_hs — the default
```

- Explicitly started by the processor
- Signals completion with a status bit, and can raise an interrupt
- The closest thing to a **function call** in hardware
- But the processor writes every input and reads every output
- Not how most production designs move data

**Our focus — it is the easiest to reason about, and every idea in it carries over.**

## Free-Running Task

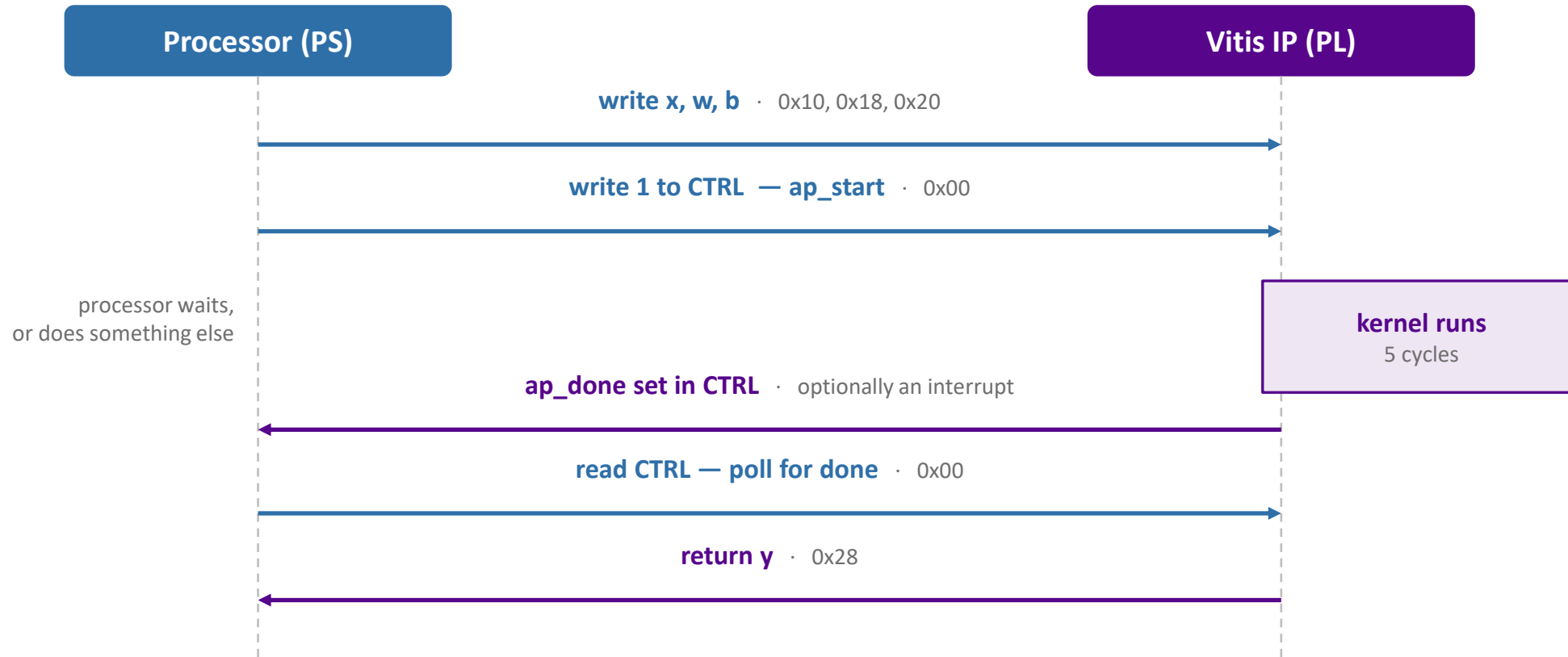
```
#pragma HLS INTERFACE ap_ctrl_none port=return  
ap_ctrl_none
```

- Runs continuously, waiting on data
- Operates without the processor once started
- Enables far greater **concurrency**
- Data arrives on a stream, not in registers
- How real accelerators are built

**Examined in the advanced course — and glimpsed next lecture, with AXI4-Stream.**



# Host-Activated Execution Model



Measured on the bus: **4 writes and 2 reads** per call, **22 cycles** of traffic for **5 cycles** of computation.

# The Demo: a Scalar Function IP

[sdrangan.github.io/hwdesign](https://sdrangan.github.io/hwdesign) — Demos › Bus Basics and Memory-Mapped Interfaces

## RUN THE WHOLE FLOW

```
cd demos/scalar_fun/scalar_fun_vitis
python scalar_fun_build.py --list-steps
python scalar_fun_build.py --through verify_csim
python scalar_fun_build.py --through report
```

## WHAT TO WATCH FOR

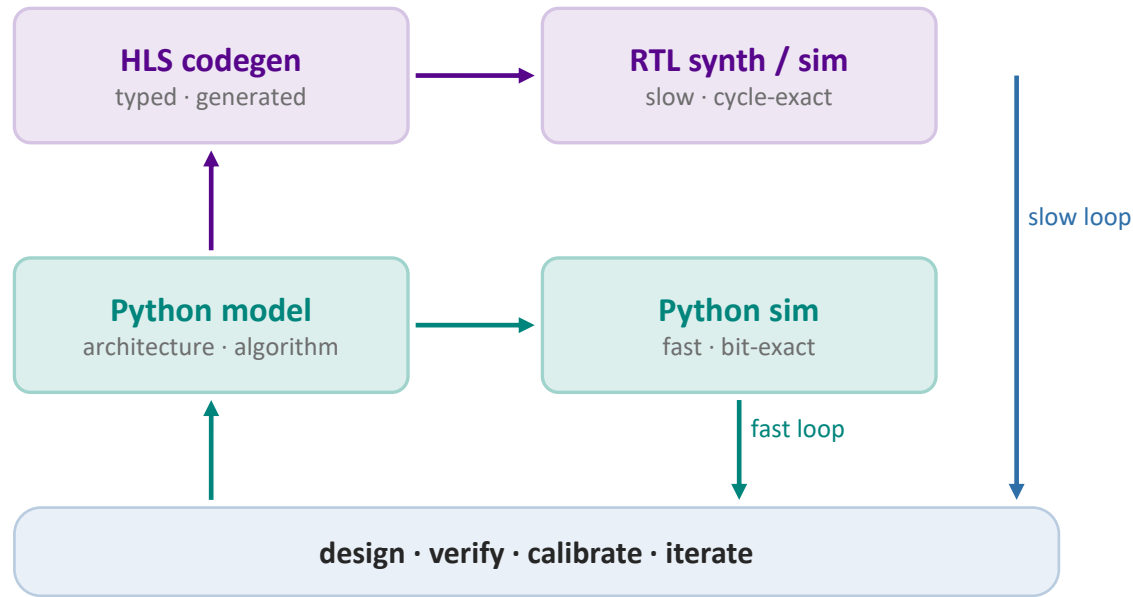
- Each step **declares what it needs** — the order is derived, not written down
- A step whose inputs have not changed is **skipped**
- The build **refuses to synthesize** code that failed its checks
- Results are checked twice: after C simulation, and again against the **synthesized RTL**

The demo ends with a timing diagram decoded from a real co-simulation — the processor writing x, w and b, starting the IP, polling for done, and reading y back.

**Everything on the next slide was measured from it.**

# Waveflow: a Python Model First

“From waveform to silicon — in Python.” Describe the datapath once, simulate it fast and bit-exact, and generate the hardware from the same source.



## THE FLOW, FOR THIS SAME IP

- 1 Model it in Python**  
the kernel, and its registers as a `VitisRegMap`
- 2 Simulate in Python**  
function, and **cycle-approximate** timing
- 3 Generate the Vitis C++**  
kernel and testbench; you keep the compute hook
- 4 C-simulation checks function**  
against the Python result
- 5 Co-simulation checks timing**  
against the Python estimate

You have already run part of it: `scalar_fun_build.py` is a Waveflow build graph. What is new above is the modelling layer.

# Waveflow vs. Native Vitis

## What it costs

- A **Python layer** between you and the tool — one more thing to learn, and to debug
- Its own vocabulary — `VitisRegMap`, `HostActivated`, `SeqTB` — before you see any hardware
- **Early-stage research software**, by its own description
- Only the **sequential** testbench lowers to Vitis: a C++ testbench is one straight-line `main()`, so a concurrent model has nothing to lower onto

## What it buys

- **One source of truth** — simulation, generated C++ and the register map all come from the same Python
- A model you can **explore in seconds**, instead of minutes of synthesis per idea
- The build **checks itself**: the Python result is the golden the C-simulation is graded against
- Timing you can **trust before synthesis**, because the estimate is checked against measured cycles

For `scalar_fun`, **nothing** — the kernel is four lines, and a model of it is pure overhead.

The payoff starts when the algorithm is worth exploring before it is worth synthesizing. [sdrangan.github.io/waveflow](https://sdrangan.github.io/waveflow)

# Limitations of AXI4-Lite for Data

**22**

cycles of bus traffic  
per call

**5**

cycles of actual  
computation

**3.7 M**

calls/s — one DSP  
does 100 M MAC/s

## IT IS SLOW TO MOVE EACH VALUE

- **Four writes and two reads** to deliver 3 values and collect 1
- About **3 cycles** per register transfer
- The bus cost does **not shrink** if the kernel gets cleverer — here the interface dominates completely

## THE PROCESSOR MUST POLL

- The host cannot know when the IP is done — it **asks, repeatedly**
- Each poll is a full AXI4-Lite read transaction
- Interrupts avoid the spinning, but cost a context switch — more than a 5-cycle kernel is worth

AXI4-Lite is not the problem — using a **control** interface as a **data** interface is.

**Next: AXI4-Stream — one transfer per cycle, no addresses, and the processor out of the inner loop.**